

Skills & Technologies Used

Market Success Viability Predictor — a plain-language guide to every tool and technique

This project combines data engineering, large language models, classical machine learning, explainable AI, full-stack web development, and cloud deployment. Below, each skill is grouped by area with a simple explanation and exactly how it was used in this project.

1. Programming & Core Tools

Skill / Tool	What it is (simply)	What we used it for
Python	A popular, readable programming language.	The entire back end: data pipeline, LLM calls, model training, and the prediction service.
JavaScript (React/JSX)	The language that makes web pages interactive.	The whole front-end application the user clicks through.
HTML & CSS	The structure and styling of web pages.	Laying out and styling the app's screens and charts.
Git & GitHub	Tools to track code changes and host code online.	Version control, collaboration, and the source the live app deploys from.

2. Machine Learning

Skill / Tool	What it is (simply)	What we used it for
XGBoost	A model that combines many small decision trees to make predictions.	Predicting the 0–100 market-viability score from a product's aspect scores.
SHAP	A method that explains which inputs pushed a prediction up or down.	Showing which aspects raise or lower each product's viability — the explainability.
scikit-learn	A standard toolkit of ML helper functions.	Cross-validation and accuracy metrics (R^2 , RMSE, AUC).
Feature engineering	Turning raw data into clean numeric inputs a model can learn from.	Aggregating thousands of review scores into per-product feature rows.
Regression modelling	Predicting a continuous number rather than a yes/no.	Estimating the viability score as a number from 0 to 100.
Model evaluation	Measuring how good a model really is, honestly.	5-fold cross-validation, per-category scores, and a held-out test set.

3. Large Language Models (LLM) & NLP

Skill / Tool	What it is (simply)	What we used it for
Large Language Models	AI that understands and generates human language (e.g., Gemini, Llama).	Reading reviews to extract aspect scores, and translating new-product specs into scores.
Prompt engineering	Carefully writing the instructions given to an LLM.	The 'null-forcing' extraction prompt and the 'bridging' prompt — the project's core prompts.
Aspect-based sentiment analysis	Scoring specific qualities of a product from text, not just overall sentiment.	Turning a review into build-quality, value, reliability, etc. scores (0–10).

Multi-provider orchestration	Rotating across several AI services to avoid rate limits.	Running ~44,000 extractions across free tiers (Gemini, Groq, Cerebras, Mistral, SambaNova).
Pydantic (validation)	A tool that enforces data has the exact expected shape.	Validating the LLM's JSON output so bad responses never corrupt the data.

4. Data Engineering

Skill / Tool	What it is (simply)	What we used it for
Data streaming / APIs	Pulling large datasets piece-by-piece instead of all at once.	Streaming the huge Amazon Reviews 2023 dataset and collecting Google Play data.
Data cleaning	Removing noise, duplicates, and bad rows.	Filtering to English 10–400-word reviews, stripping HTML, and de-duplicating.
pandas & NumPy	The standard Python libraries for tables and numbers.	Aggregating reviews to product level and building the feature matrix.
Parquet / PyArrow	An efficient file format for large tables.	Storing the model's feature matrix compactly and quickly.
Statistical filtering	Using simple statistics to flag suspicious patterns.	Fake-review detection: review bursts, extreme rating shapes, and near-duplicate text.
Data contracts / validation	Guaranteeing data matches an agreed shape between stages.	A manifest + loader that refuses to run on invalid data — so stages never silently misalign.

5. Web Development

Skill / Tool	What it is (simply)	What we used it for
React	A framework for building interactive user interfaces.	The Simulator, Spec Coach, charts, and result cards.
Vite	A fast build tool that compiles the front end.	Bundling the React code into files the browser can load.
FastAPI	A modern Python framework for building web APIs.	Serving predictions and the app itself to the browser.
Uvicorn	A lightweight web server for Python apps.	Actually running the FastAPI back end.
REST API design	A standard way for the front end and back end to talk.	The 'api' endpoints the UI calls to get categories and predictions.
Data visualization	Turning numbers into clear charts.	The viability donut, aspect bar chart, market-positioning grid, and SHAP cards.

6. Deployment & DevOps

Skill / Tool	What it is (simply)	What we used it for
Docker	Packaging an app so it runs identically anywhere.	Containerizing the whole system into one deployable image.
Render (cloud hosting)	A platform that runs your app on the internet.	Hosting the live, public web app for free.
Secrets management	Keeping passwords and API keys out of the code.	API keys live only in environment variables / gitignored config — never in the repo.
Reproducibility	Making results repeatable by others.	Fixed random seeds and cached, re-runnable pipeline stages.

7. Concepts & Techniques Demonstrated

Skill / Tool	What it is (simply)	What we used it for
--------------	---------------------	---------------------

Explainable AI (XAI)	Making a model's decisions transparent, not a black box.	Every prediction comes with a per-aspect breakdown of why.
Confidence & abstention	Knowing when the model should NOT answer.	Low-confidence specs get a warning; near-empty specs have the prediction withheld.
Category-scoped modelling	Judging a product only against its own category's norms.	A smartphone is judged vs smartphones, not vs power banks.
Honest evaluation	Reporting real limitations instead of only the best number.	Leading with the conservative 'aspects-only' score and disclosing the proxy-label caveats.

In short: this project is a full pipeline — collect and clean real data, use LLMs to extract structured meaning, train and explain a machine-learning model, wrap it in a web app, and deploy it live — with honesty about what it can and cannot do.